Project Setup

Setting up a well-structured project is crucial for specification-driven development. This exercise guides you through creating a comprehensive project structure that supports both specifications and implementation.

Exercise 1.1: Create Project Structure

Create a new folder structure for your chatbot specification project:

```
Create a new project folder structure for our serverless MyCoolChatbot-KiroSpec application:

**Project Name**: `MyCoolChatbotSpec`

**Required Folder Structure**:
```
MyCoolChatbotSpec/
├── .kiro/
│ ├── settings/
│ │ └── mcp.json
│ └── steering/
├── specifications/
│ ├── foundation/
│ ├── infrastructure/
│ ├── frontend/
│ ├── backend/
│ └── security/
├── implementation/
│ ├── infrastructure/
│ ├── frontend/
│ └── backend/
├── docs/
│ ├── architecture/
│ ├── api/
│ └── deployment/
└── README.md
```

Create this structure and initialize each folder with appropriate placeholder files. Include a comprehensive README.md that explains the project purpose, struct
```

Expected Outcome

- Complete project folder structure created
- README.md with project overview and navigation guide
- Placeholder files in each specification folder
- Clear separation between specifications and implementation

Exercise 1.2: Initialize Kiro Spec Mode

Set up Kiro for specification-driven development:

```
Initialize Kiro in Spec mode for our chatbot project:

1. Open the `MyCoolChatbotSpec` folder in Kiro
2. Navigate to the "Let's build" page
3. Select "Spec" instead of "Vibe" mode
4. Create a new spec with the following initial structure:

**Spec Configuration**:
- **Project Type**: Serverless Web Application
- **Primary Technologies**: AWS (Lambda, API Gateway, S3, CloudFront, Bedrock)
- **Frontend Framework**: React with TypeScript
- **Infrastructure**: AWS CDK
- **Development Approach**: Specification-driven with comprehensive documentation

Initialize the spec and create the initial `tasks.md` file that will track our progress throughout the workshop.
```

Expected Outcome

- Kiro configured in Spec mode
- Initial spec created with project metadata
- `tasks.md` file initialized for progress tracking
- Spec interface ready for detailed specification creation

Exercise 1.3: Project Configuration and Standards

Define project-wide standards and configurations:

```
Create comprehensive project configuration and standards documentation:

**Configuration Requirements**:
- **Naming Conventions**: Define consistent naming for files, folders, AWS resources, and code elements
- **Documentation Standards**: Specify format, structure, and requirements for all documentation
- **Code Standards**: Define coding conventions for TypeScript, React, and AWS CDK
- **Version Control**: Establish branching strategy and commit message conventions
- **Quality Gates**: Define acceptance criteria and quality standards for each phase

**Deliverables**:
1. `PROJECT_STANDARDS.md` - Comprehensive standards document
2. `.gitignore` - Appropriate exclusions for the project
3. `CONTRIBUTING.md` - Guidelines for team collaboration
4. `CHANGELOG.md` - Template for tracking changes

Include specific examples and templates that team members can follow throughout the project.
```

Expected Outcome

- Comprehensive project standards documentation
- Clear guidelines for naming, coding, and documentation
- Version control configuration and guidelines
- Quality gates and acceptance criteria defined

Exercise 1.4: Development Environment Setup

Configure the development environment and tooling:

```
Set up the development environment for our chatbot project:

**Development Tools Configuration**:
- **Node.js and npm**: Specify required versions and configuration
- **AWS CLI**: Configure for local development and testing
- **TypeScript**: Set up compiler configuration and type definitions
- **Testing Framework**: Configure Jest and testing utilities
- **Linting and Formatting**: Set up ESLint, Prettier, and pre-commit hooks

**Environment Files**:
1. `package.json` - Project dependencies and scripts
2. `tsconfig.json` - TypeScript configuration
3. `.eslintrc.js` - Linting rules and configuration
4. `.prettierrc` - Code formatting rules
5. `jest.config.js` - Testing configuration

**Development Scripts**:
- Build and development server scripts
- Testing and quality assurance scripts
- Deployment and infrastructure scripts
- Documentation generation scripts

Ensure all configurations align with the project standards defined in the previous exercise.
```

Expected Outcome

- Complete development environment configuration
- All necessary tooling configured and documented
- Development scripts for common tasks
- Environment ready for specification creation and implementation

Key Takeaways

After completing the Project Setup exercises, you should have:

1. **Well-Organized Structure**: Clear separation between specifications, implementation, and documentation

aws workshop studio
rajeshkumar
Event ends in 2 days 8 hours 11 minutes.

Next Steps

With your project foundation established, you're ready to configure the advanced Kiro features that will enhance your development workflow throughout the specification and implementation phases.

The next section will guide you through setting up MCP servers, Agent Hooks, and Agent Steering to create a powerful, automated development environment.

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Privacy policy Terms of use Cookie preferences